AI-Enabled NIDS Backend Demo

Cyber-Modern Classroom Evaluation Deck

Presenter Details

Name: _____
Roll No: _____
Class/Section: _____
Date: _____

Demo Snapshot

Duration: 10 to 15 minutes
Mode: Live API + terminal workflow
Core focus: Auth, sensors, audit, AI analytics
Target: Clear, defensible security architecture

Demo Goals and Outcomes

What the evaluator should see clearly

Goals

Demonstrate reliable startup and readiness checks
Prove authentication and RBAC with Keycloak-backed JWT
Walk through sensor onboarding lifecycle
Show tamper-aware audit visibility
Show operational security checks and anomaly interfaces

Evaluator Outcomes

System is healthy and production-shaped
Access control is role enforced, not just login based
Sensor enrollment uses single-use key flow
Critical actions appear in audit logs
Security monitoring endpoints are present and queryable

What Is Being Demonstrated

Core capabilities in this session

Identity and Access

Keycloak login/session flow
JWT claims and role checks

Sensor Lifecycle

Create sensor
Enroll with single-use key
Activate

Audit Pipeline

Audit events recorded
Query recent logs

Security Operations

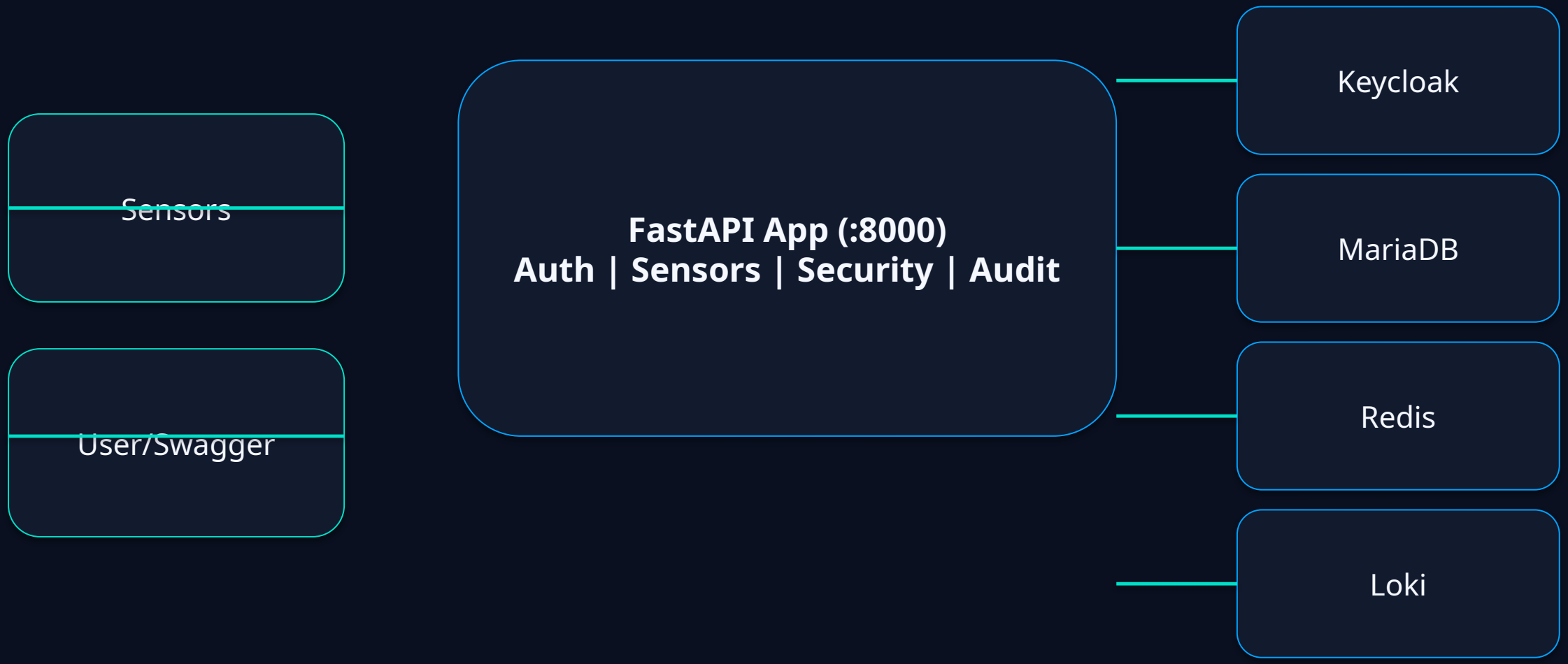
Run rotation health checks
Review findings

AI Analytics Surface

Inspect anomaly and risk-score endpoints

Architecture Overview

FastAPI core with security-first service mesh



Private network: MariaDB, Redis, Loki, step-ca, Infisical | Frontend network: App, Keycloak

Defense-in-Depth and RBAC Roles

Layered controls and bounded permissions

Security Layers

Layer 1: Keycloak SSO, MFA-ready auth, role validation
Layer 2: Sensor onboarding via single-use enrollment keys
Layer 3: Audit logging to DB and Loki pipeline
Layer 4: Rotation health checks and anomaly endpoints
Layer 5: Optional mTLS + ZTNA enforcement controls

Role Summary

super_admin
security_analyst
auditor
sensor_manager

Super admin: full control + rotation checks
Security analyst: anomaly + risk views
Auditor: read audit evidence
Sensor manager: sensor lifecycle operations

Demo Setup Checklist

Pre-demo reliability steps

Environment and Prerequisites

Install: Docker, Docker Compose, curl, jq
Copy env: cp .env.example .env
Required: DB_USER=nids
Required: KEYCLOAK_CLIENT_SECRET=change_me_client_secret
Use clean startup before evaluation day

Clean Startup Commands

```
docker compose down -v
docker compose up -d --build

docker compose ps
curl -sS http://localhost:8000/health/ready |
jq

Open:
http://localhost:8000/docs
http://localhost:8080
```

Live Demo Flow Timeline

Nine steps from readiness to monitoring

1. Verify /health/ready
2. POST /auth/login
3. GET /auth/me
4. POST /sensors/
5. POST /sensors/{id}/enroll
6. POST /sensors/activate
7. GET /sensors/
8. GET /audit/logs?limit=5
9. POST /security/rotation-health/run

Narrative Anchor

Authenticate -> Authorize -> Provision -> Activate -> Observe -> Validate Security Health

Key Command Highlights

Concise commands on the main flow

```
curl -sS http://localhost:8000/health/ready | jq
```

```
TOKEN=$(curl -sS -X POST http://localhost:8000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"Admin@nids2024!"}' | jq -r '.access_token')
```

```
curl -sS -X POST http://localhost:8000/sensors/ \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"name":"demo-sensor","network_segment":"172.28.10.0/24"}'
```

Proof of Results

What successful outputs look like

Validation Evidence

GET /auth/me shows role claim: super_admin
Sensor status progression: pending -> active
GET /sensors/ returns updated inventory and totals
GET /audit/logs returns latest event types
POST /security/rotation-health/run returns counts

Expected Signals

```
"status": "ready"
"roles": ["super_admin"]
"latest_event": "sensor_activated"
"critical": <n>, "warning": <n>
```

Evaluator should see evidence, not only claims.
Every key action has matching system output.

AI and Monitoring Endpoints

Security analytics visibility

Endpoints to show in Swagger

```
GET /security/anomalies
GET /security/anomalies/risk-score
POST /security/rotation-health/run
```

How to explain AI in demo

Anomaly service evaluates event streams
Risk scoring synthesizes weighted signals
Endpoints expose current analytic state
Demonstrates AI-backed security operations

Monitoring posture

Operational health checks for rotations
Findings classified by severity
Early warning before cert/secret failures
Works with audit trail for incident response

Common Issues and Quick Fixes

Demo-day recovery playbook

App not starting	Check .env values (DB_USER=nids and Keycloak secret); run clean restart
/auth/login returns 423 Locked	Clear lockout keys in Redis for admin before login retry
Services unhealthy	Use compose status and logs for keycloak/mariadb/redis/loki
step-ca warning in rotation check	Expected if STEP_CA_FINGERPRINT unset and mTLS enrollment not shown

```
docker compose logs --tail=100 app
docker compose ps
docker compose logs --tail=150 keycloak mariadb redis loki
```

Appendix: Full Live Demo Script

Copy/paste fallback from Demo.md

```
cd ~/NIDS

# 0) Optional safety: clear admin lockout state
source .env
docker exec nids-redis sh -lc \
  "REDISCLI_AUTH='$REDIS_PASSWORD' redis-cli DEL lockout:temp:admin lockout:failures:admin
lockout:perm:admin"

# 1) Show system readiness
curl -sS http://localhost:8000/health/ready | jq

# 2) Login as seeded admin user from realm import
TOKEN=$(curl -sS -X POST http://localhost:8000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"Admin@nids2024!"}' | jq -r '.access_token')

echo "Token length: ${#TOKEN}"

# 3) Show identity and RBAC role
curl -sS http://localhost:8000/auth/me \
  -H "Authorization: Bearer $TOKEN" | jq

# 4) Create a sensor
SENSOR_NAME="demo-sensor-$(date +%s)"
SENSOR=$(curl -sS -X POST http://localhost:8000/sensors/ \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d "{\"name\":\"$SENSOR_NAME\",\"network_segment\":\"172.28.10.0/24\"}")
```

```
echo "$SENSOR" | jq

SENSOR_ID=$(echo "$SENSOR" | jq -r '.id')

# 5) Enroll sensor (single-use API key)
ENROLL=$(curl -sS -X POST http://localhost:8000/sensors/$SENSOR_ID/enroll \
  -H "Authorization: Bearer $TOKEN")
echo "$ENROLL" | jq

API_KEY=$(echo "$ENROLL" | jq -r '.api_key')

# 6) Activate sensor using enrollment key
curl -sS -X POST http://localhost:8000/sensors/activate \
  -H 'Content-Type: application/json' \
  -d "{\"api_key\":\"$API_KEY\"}" | jq

# 7) Show sensor inventory count
curl -sS http://localhost:8000/sensors/ \
  -H "Authorization: Bearer $TOKEN" | jq '{total, sensors: [.sensors[].status]}'

# 8) Show recent audit logs
curl -sS "http://localhost:8000/audit/logs?limit=5" \
  -H "Authorization: Bearer $TOKEN" | jq '{total, latest_event: .logs[0].event_type}'

# 9) Trigger security rotation check
curl -sS -X POST http://localhost:8000/security/rotation-health/run \
  -H "Authorization: Bearer $TOKEN" | jq '{total,critical,warning}'
```